

Library Management System — Data Analysis with Python (Pandas)

Level: Intermediate **Tools:** Python (Anaconda), pandas

Project Overview

This project builds and analyzes a Library Management System dataset using Python. It covers loading real data, performing CRUD-style operations (Create, Read, Update, Delete), building summary tables, and running analytical queries to answer real business questions about branches, employees, members, books, and book-loan activity.

Objectives

1. **Load and explore the library data** — six related tables: `branch`, `employees`, `members`, `books`, `issued_status`, `return_status`.
2. **CRUD operations** — add, update, delete, and retrieve records using pandas.
3. **Derived summary tables** — build new tables from existing data (similar in spirit to a database "CREATE TABLE AS SELECT").
4. **Data analysis** — answer 20 specific business questions about the library's operations.

We'll go step by step — each step explains *what* we're doing and *why*, then shows the code, then looks at the result. New pandas concepts are explained the first time they show up.

Step 0 — Import the Libraries We Need

We only need a handful of tools for this project:

- **pandas** — for loading and manipulating tabular data (our main tool throughout)
- **numpy** — a few numeric helpers
- **datetime** — for working with dates (issue dates, due dates, etc.)

```
In [1]: import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Show more rows/columns in outputs so tables aren't cut off
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 160)

# A single "today" reference we'll reuse anywhere we need the current date
today = pd.Timestamp.now().normalize()

print("Libraries loaded. pandas version:", pd.__version__)
print("Today's date reference:", today.date())
```

```
Libraries loaded. pandas version: 3.0.2
Today's date reference: 2026-09-15
```

Step 1 — Load the Data

Our library data lives in six CSV files, one per table. We load each one with `pd.read_csv()`, which reads a CSV file straight into a **DataFrame** — pandas' version of a spreadsheet/table.

We load them one at a time so we can look at each before moving on.

1.1 Branches

```
In [2]: branch = pd.read_csv("branch.csv")
print(branch.shape)  # (rows, columns)
branch
(5, 4)
```

```
Out[2]:
```

	branch_id	manager_id	branch_address	contact_no
0	B001	E109	123 Main St	919099988676
1	B002	E109	456 Elm St	919099988677
2	B003	E109	789 Oak St	919099988678
3	B004	E110	567 Pine St	919099988679
4	B005	E110	890 Maple St	919099988680

1.2 Employees

```
In [3]: employees = pd.read_csv("employees.csv")
print(employees.shape)
employees
```

(11, 5)

```
Out[3]:
```

	emp_id	emp_name	position	salary	branch_id
0	E101	John Doe	Clerk	60000.0	B001
1	E102	Jane Smith	Clerk	45000.0	B002
2	E103	Mike Johnson	Librarian	55000.0	B001
3	E104	Emily Davis	Assistant	40000.0	B001
4	E105	Sarah Brown	Assistant	42000.0	B001
5	E106	Michelle Ramirez	Assistant	43000.0	B001
6	E107	Michael Thompson	Clerk	62000.0	B005
7	E108	Jessica Taylor	Clerk	46000.0	B004
8	E109	Daniel Anderson	Manager	57000.0	B003
9	E110	Laura Martinez	Manager	41000.0	B005
10	E111	Christopher Lee	Assistant	65000.0	B005

1.3 Members

```
In [4]: members = pd.read_csv("members.csv")
print(members.shape)
members.head()
```

(12, 4)

```
Out[4]:
```

	member_id	member_name	member_address	reg_date
0	C101	Alice Johnson	123 Main St	2021-05-15
1	C102	Bob Smith	456 Elm St	2021-06-20
2	C103	Carol Davis	789 Oak St	2021-07-10
3	C104	Dave Wilson	567 Pine St	2021-08-05
4	C105	Eve Brown	890 Maple St	2021-09-25

1.4 Books

```
In [5]: books = pd.read_csv("books.csv")
print(books.shape)
books.head()
```

(35, 7)

Out[5]:

	isbn	book_title	category	rental_price	status	author	publisher
0	978-0-553-29698-2	The Catcher in the Rye	Classic	7.0	yes	J.D. Salinger	Little, Brown and Company
1	978-0-330-25864-8	Animal Farm	Classic	5.5	yes	George Orwell	Penguin Books
2	978-0-14-118776-1	One Hundred Years of Solitude	Literary Fiction	6.5	yes	Gabriel Garcia Marquez	Penguin Books
3	978-0-525-47535-5	The Great Gatsby	Classic	8.0	yes	F. Scott Fitzgerald	Scribner
4	978-0-141-44171-6	Jane Eyre	Classic	4.0	yes	Charlotte Bronte	Penguin Classics

1.5 Issued Status (which member borrowed which book, from which employee, and when)

```
In [6]: issued_status = pd.read_csv("issued_status.csv")
print(issued_status.shape)
issued_status.head()
```

(35, 6)

Out[6]:

	issued_id	issued_member_id	issued_book_name	issued_date	issued_book_isbn	issued_emp_id
0	IS106	C106	Animal Farm	2024-03-10	978-0-330-25864-8	E104
1	IS107	C107	One Hundred Years of Solitude	2024-03-11	978-0-14-118776-1	E104
2	IS108	C108	The Great Gatsby	2024-03-12	978-0-525-47535-5	E104
3	IS109	C109	Jane Eyre	2024-03-13	978-0-141-44171-6	E105
4	IS110	C110	The Alchemist	2024-03-14	978-0-307-37840-1	E105

1.6 Return Status (records of books coming back)

```
In [7]: return_status = pd.read_csv("return_status.csv")
print(return_status.shape)
return_status.head()
```

(18, 5)

Out[7]:

	return_id	issued_id	return_book_name	return_date	return_book_isbn
0	RS101	IS101	NaN	2023-06-06	NaN
1	RS102	IS105	NaN	2023-06-07	NaN
2	RS103	IS103	NaN	2023-08-07	NaN
3	RS104	IS106	NaN	2024-05-01	NaN
4	RS105	IS107	NaN	2024-05-03	NaN

Step 2 — Explore the Data (a quick health check)

Before analyzing anything, it's good practice to check:

- What columns and data types do we have? → `.info()`
- Are there missing values? → `.isnull().sum()`

Let's check the `books` table as an example.

```
In [8]: books.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   isbn             35 non-null    str
1   book_title       35 non-null    str
2   category         35 non-null    str
3   rental_price     35 non-null    float64
4   status           35 non-null    str
5   author           35 non-null    str
6   publisher        35 non-null    str
dtypes: float64(1), str(6)
memory usage: 2.0 KB
```

```
In [9]: books.isnull().sum()
```

```
Out[9]: isbn             0
book_title             0
category               0
rental_price           0
status                 0
author                 0
publisher              0
dtype: int64
```

🔗 **Concept — .info() and .isnull().sum() :** .info() gives you the column names, how many non-null values each has, and their data type (object = text, float64 / int64 = numbers). .isnull().sum() counts missing values per column — useful for spotting data-quality issues early.

Let's do the same quick check on `issued_status` and `return_status` , since those drive most of our analysis.

```
In [10]: issued_status.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   issued_id       35 non-null    str
1   issued_member_id 35 non-null    str
2   issued_book_name 35 non-null    str
3   issued_date      35 non-null    str
4   issued_book_isbn 35 non-null    str
5   issued_emp_id    35 non-null    str
dtypes: str(6)
memory usage: 1.8 KB
```

```
In [11]: return_status.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 18 entries, 0 to 17
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   return_id       18 non-null    str
1   issued_id       18 non-null    str
2   return_book_name 0 non-null     float64
3   return_date     18 non-null    str
4   return_book_isbn 0 non-null     float64
dtypes: float64(2), str(3)
memory usage: 852.0 bytes
```

One thing to fix before we go further: `issued_date` and `return_date` are currently plain text (object), not real dates. We need proper dates so we can do date math later (like "how many days overdue?"). This is a general structural fix — it doesn't belong to any one question, it just makes the tables usable, so we do it now for every table that has a date column.

(Note: we're **not** touching `return_status` beyond this — we'll come back and add whatever extra info a question actually needs, exactly when that question asks for it, instead of guessing upfront.)

Step 2.1 — Convert date columns to real dates

`pd.to_datetime()` converts a text column into pandas' datetime type, which unlocks date arithmetic (subtracting dates, filtering by "last N days", etc.).

```
In [12]: issued_status["issued_date"] = pd.to_datetime(issued_status["issued_date"])
return_status["return_date"] = pd.to_datetime(return_status["return_date"])
members["reg_date"] = pd.to_datetime(members["reg_date"])
```

```
issued_status.dtypes
```

```
Out[12]: issued_id          str
         issued_member_id   str
         issued_book_name   str
         issued_date        datetime64[us]
         issued_book_isbn   str
         issued_emp_id      str
         dtype: object
```

Now our six tables are loaded and the dates are usable. Let's move on to the CRUD operations — we'll deal with anything else the data is missing only when a specific task actually needs it.

Step 3 — CRUD Operations (Create, Read, Update, Delete)

These are the four basic operations any data system needs to support. We'll do each one on our tables.

Task 1 — Create: Add a New Book Record

Goal: Add *"To Kill a Mockingbird"* by Harper Lee to the `books` table.

 **Concept — adding a row:** build a one-row DataFrame with the same column names, then glue it onto the bottom of the existing table with `pd.concat()`. `ignore_index=True` re-numbers the rows afterward so the index stays clean (0, 1, 2, ...).

```
In [13]: new_book = pd.DataFrame([{
         "isbn": "978-1-60129-456-2",
         "book_title": "To Kill a Mockingbird",
         "category": "Classic",
         "rental_price": 6.00,
         "status": "yes",
         "author": "Harper Lee",
         "publisher": "J.B. Lippincott & Co."
       }])

books = pd.concat([books, new_book], ignore_index=True)

print("Books table now has", len(books), "rows")
books.tail(3)
```

Books table now has 36 rows

```
Out[13]:
```

	isbn	book_title	category	rental_price	status	author	publisher
33	978-0-7432-4722-5	Angels & Demons	Mystery	7.5	yes	Dan Brown	Doubleday
34	978-0-7432-7356-4	The Hobbit	Fantasy	7.0	yes	J.R.R. Tolkien	Houghton Mifflin Harcourt
35	978-1-60129-456-2	To Kill a Mockingbird	Classic	6.0	yes	Harper Lee	J.B. Lippincott & Co.

Task 2 — Update: Change an Existing Member's Address

Goal: Update member `C103` 's address to `'125 Oak St'`.

 **Concept — `.loc[row_filter, column] = value`:** `.loc` lets you select rows *and* a column at the same time and assign a new value — the pandas way of doing a targeted `UPDATE`.

```
In [14]: members.loc[members["member_id"] == "C103", "member_address"] = "125 Oak St"

members[members["member_id"] == "C103"]
```

```
Out[14]:
```

	member_id	member_name	member_address	reg_date
2	C103	Carol Davis	125 Oak St	2021-07-10

Task 3 — Delete: Remove a Record from Issued Status

Goal: Remove the loan record with `issued_id = 'IS140'`.

 **Concept — deleting rows:** there's no `.delete()` in pandas. Instead, you keep everything that does **not** match your condition, using the `~` (NOT) operator to flip a boolean mask.

```
In [15]: before = len(issued_status)
         issued_status = issued_status[~(issued_status["issued_id"] == "IS140")].reset_index(drop=True)
         after = len(issued_status)
```

```
print(f"Rows before: {before}, after: {after}")
```

Rows before: 35, after: 34

Task 4 — Read: All Books Issued by a Specific Employee

Goal: Find every loan processed by employee `E106`.

```
In [16]: task4 = issued_status[issued_status["issued_emp_id"] == "E106"]
task4
```

```
Out[16]:
```

	issued_id	issued_member_id	issued_book_name	issued_date	issued_book_isbn	issued_emp_id
6	IS112	C109	A Game of Thrones	2024-03-16	978-0-09-957807-9	E106
7	IS113	C109	A Peoples History of the United States	2024-03-17	978-0-393-05081-8	E106
8	IS114	C109	The Guns of August	2024-03-18	978-0-19-280551-1	E106
26	IS132	C106	The Hobbit	2024-04-05	978-0-7432-7356-4	E106
27	IS133	C107	Angels & Demons	2024-04-06	978-0-7432-4722-5	E106
28	IS134	C107	The Diary of a Young Girl	2024-04-07	978-0-375-41398-8	E106

Task 5 — Members Who Have Issued More Than One Book

Goal: Find every member who has borrowed more than one book in total.

💡 **Concept — `groupby()`** : this is one of the most important pandas tools. It splits your data into groups (here, one group per `issued_member_id`), then lets you compute something *per group* — like a count, sum, or average. After aggregating, we filter the *result* — pandas has no direct equivalent of SQL's `HAVING`, so you just apply a normal filter after the `groupby`.

```
In [17]: books_per_member = (
    issued_status.groupby("issued_member_id")
                .size() # count rows in each group
                .reset_index(name="books_issued")
)

task5 = books_per_member[books_per_member["books_issued"] > 1]
task5.sort_values("books_issued", ascending=False)
```

```
Out[17]:
```

	issued_member_id	books_issued
8	C109	7
6	C107	6
4	C105	5
9	C110	5
5	C106	4
1	C102	2
7	C108	2

Step 4 — Building Derived Summary Tables

Sometimes the most useful output of an analysis isn't a single number — it's a whole new table you can reuse elsewhere (save to CSV, feed into a chart, etc.). We'll build a few of those here.

Task 6 — Total Times Each Book Has Been Issued

Goal: For every book, count how many times it has been issued, and save that as its own table.

💡 **Concept — `merge()`** : this is pandas' version of a SQL `JOIN` — it combines two tables based on a shared key (here, ISBN). We merge `issued_status` with `books` so each loan record also carries the book's title.

```
In [18]: book_issue_counts = (
    issued_status.merge(books, left_on="issued_book_isbn", right_on="isbn")
                .groupby(["isbn", "book_title"])
                .agg(issue_count=("issued_id", "count"))
                .reset_index()
                .sort_values("issue_count", ascending=False)
```

```
)  
book_issue_counts
```

Out[18]:

	isbn	book_title	issue_count
26	978-0-679-76489-8	Harry Potter and the Sorcerers Stone	2
22	978-0-525-47535-5	The Great Gatsby	2
0	978-0-06-025492-6	Where the Wild Things Are	1
1	978-0-06-112008-4	To Kill a Mockingbird	1
4	978-0-09-957807-9	A Game of Thrones	1
5	978-0-14-027526-3	A Tale of Two Cities	1
2	978-0-06-112241-5	The Kite Runner	1
3	978-0-06-440055-8	Charlotte's Web	1
8	978-0-14-143951-8	Pride and Prejudice	1
9	978-0-141-44171-6	Jane Eyre	1
10	978-0-19-280551-1	The Guns of August	1
11	978-0-307-37840-1	The Alchemist	1
12	978-0-307-58837-1	Sapiens: A Brief History of Humankind	1
13	978-0-330-25864-8	Animal Farm	1
6	978-0-14-044930-3	The Histories	1
7	978-0-14-118776-1	One Hundred Years of Solitude	1
15	978-0-375-41398-8	The Diary of a Young Girl	1
14	978-0-345-39180-3	Dune	1
18	978-0-393-91257-8	Guns, Germs, and Steel: The Fates of Human Soc...	1
16	978-0-385-33312-0	The Shining	1
19	978-0-451-52993-5	Fahrenheit 451	1
20	978-0-451-52994-2	Moby Dick	1
21	978-0-452-28240-7	Brave New World	1
17	978-0-393-05081-8	A Peoples History of the United States	1
23	978-0-553-29698-2	The Catcher in the Rye	1
24	978-0-670-81302-4	The Road	1
25	978-0-679-64115-3	1984	1
27	978-0-679-77644-3	Beloved	1
28	978-0-7432-4722-5	Angels & Demons	1
29	978-0-7432-7356-4	The Hobbit	1
30	978-0-7432-7357-1	1491: New Revelations of the Americas Before C...	1
31	978-0-7434-7679-3	The Stand	1

Step 5 — Data Analysis: Answering Business Questions

Now for the core analysis — a series of specific questions the library wants answered.

Task 7 — All Books in a Specific Category

Goal: List every book in the 'Classic' category.

```
In [19]: task7 = books[books["category"] == "Classic"]  
task7
```

Out[19]:

	isbn	book_title	category	rental_price	status	author	publisher
0	978-0-553-29698-2	The Catcher in the Rye	Classic	7.0	yes	J.D. Salinger	Little, Brown and Company
1	978-0-330-25864-8	Animal Farm	Classic	5.5	yes	George Orwell	Penguin Books
3	978-0-525-47535-5	The Great Gatsby	Classic	8.0	yes	F. Scott Fitzgerald	Scribner
4	978-0-141-44171-6	Jane Eyre	Classic	4.0	yes	Charlotte Bronte	Penguin Classics
17	978-0-14-143951-8	Pride and Prejudice	Classic	5.0	yes	Jane Austen	Penguin Classics
28	978-0-14-027526-3	A Tale of Two Cities	Classic	4.5	yes	Charles Dickens	Penguin Books
30	978-0-451-52994-2	Moby Dick	Classic	6.5	yes	Herman Melville	Penguin Books
31	978-0-06-112008-4	To Kill a Mockingbird	Classic	5.0	yes	Harper Lee	J.B. Lippincott & Co.
35	978-1-60129-456-2	To Kill a Mockingbird	Classic	6.0	yes	Harper Lee	J.B. Lippincott & Co.

Task 8 — Total Rental Income by Category

Goal: For each book category, add up the rental income generated (rental price × number of times issued).

We reuse the same `merge()` + `groupby()` pattern from Task 6, just aggregating a different column.

In [20]:

```
income_by_category = (
    books.merge(issued_status, left_on="isbn", right_on="issued_book_isbn")
    .groupby("category")["rental_price"]
    .sum()
    .reset_index(name="total_rental_income")
    .sort_values("total_rental_income", ascending=False)
)

income_by_category
```

Out[20]:

	category	total_rental_income
1	Classic	53.5
5	History	49.5
3	Fantasy	28.5
2	Dystopian	25.5
4	Fiction	14.5
6	Horror	13.0
9	Science Fiction	8.5
0	Children	7.5
8	Mystery	7.5
7	Literary Fiction	6.5

Task 9 — Members Who Registered Recently

Goal: List members who registered in the last 180 days.

Let's just write the filter directly — compare `reg_date` against today's date minus 180 days.

In [21]:

```
recent_cutoff = today - pd.Timedelta(days=180)
task9 = members[members["reg_date"] >= recent_cutoff]
task9
```

Out[21]:

member_id	member_name	member_address	reg_date
-----------	-------------	----------------	----------

The result is **empty**. Before assuming the code is wrong, let's check the data itself — when did our members actually register?

In [22]:

members["reg_date"].agg(["min", "max"])

Out[22]:

```
min    2021-05-15
max    2024-06-01
Name: reg_date, dtype: datetime64[us]
```

That explains it: every registration in this dataset is from 2021–2024, so none of them fall within 180 days of *today's real date*. The filter logic is correct — the data just doesn't have anything recent in it.

In a live library system this wouldn't be an issue (new members register every week). To actually demonstrate the query working, let's add two new members the way a librarian would when someone signs up today — using the same "add a row" technique from Task 1 — and then re-run the same filter.

In [23]:

```
new_members = pd.DataFrame([
    {"member_id": "C120", "member_name": "Liam Carter", "member_address": "145 Main St", "reg_date": today - ti
    {"member_id": "C121", "member_name": "Maya Chen", "member_address": "12 Birch Ave", "reg_date": today - t
])
members = pd.concat([members, new_members], ignore_index=True)

task9 = members[members["reg_date"] >= recent_cutoff]
task9
```

Out[23]:

	member_id	member_name	member_address	reg_date
12	C120	Liam Carter	145 Main St	2026-07-17
13	C121	Maya Chen	12 Birch Ave	2026-08-26

Task 10 — Employees with Their Branch Manager's Name and Branch Details

Goal: For every employee, show their branch's address/contact info *and* their manager's name.

🔗 **Concept — a "self-merge":** `branch.manager_id` points to another row in `employees`. To pull in the manager's name, we merge `employees` against *itself* — once as the employee, once (renamed) as the manager.

In [24]:

```
managers = employees[["emp_id", "emp_name"]].rename(
    columns={"emp_id": "manager_id", "emp_name": "manager_name"}
)

task10 = (
    employees.merge(branch, on="branch_id")
    .merge(managers, on="manager_id")
)

task10[["emp_id", "emp_name", "position", "salary", "branch_id",
        "branch_address", "contact_no", "manager_name"]]
```

Out[24]:

	emp_id	emp_name	position	salary	branch_id	branch_address	contact_no	manager_name
0	E101	John Doe	Clerk	60000.0	B001	123 Main St	919099988676	Daniel Anderson
1	E102	Jane Smith	Clerk	45000.0	B002	456 Elm St	919099988677	Daniel Anderson
2	E103	Mike Johnson	Librarian	55000.0	B001	123 Main St	919099988676	Daniel Anderson
3	E104	Emily Davis	Assistant	40000.0	B001	123 Main St	919099988676	Daniel Anderson
4	E105	Sarah Brown	Assistant	42000.0	B001	123 Main St	919099988676	Daniel Anderson
5	E106	Michelle Ramirez	Assistant	43000.0	B001	123 Main St	919099988676	Daniel Anderson
6	E107	Michael Thompson	Clerk	62000.0	B005	890 Maple St	919099988680	Laura Martinez
7	E108	Jessica Taylor	Clerk	46000.0	B004	567 Pine St	919099988679	Laura Martinez
8	E109	Daniel Anderson	Manager	57000.0	B003	789 Oak St	919099988678	Daniel Anderson
9	E110	Laura Martinez	Manager	41000.0	B005	890 Maple St	919099988680	Laura Martinez
10	E111	Christopher Lee	Assistant	65000.0	B005	890 Maple St	919099988680	Laura Martinez

Task 11 — Books Priced Above a Threshold

Goal: Build a table of "premium" books that rent for more than \$7.00.

In [25]:

```
expensive_books = books[books["rental_price"] > 7.00].sort_values("rental_price", ascending=False)
expensive_books
```

Out[25]:

	isbn	book_title	category	rental_price	status	author	publisher
9	978-0-393-05081-8	A Peoples History of the United States	History	9.0	yes	Howard Zinn	Harper Perennial
22	978-0-345-39180-3	Dune	Science Fiction	8.5	yes	Frank Herbert	Ace
3	978-0-525-47535-5	The Great Gatsby	Classic	8.0	yes	F. Scott Fitzgerald	Scribner
11	978-0-307-58837-1	Sapiens: A Brief History of Humankind	History	8.0	no	Yuval Noah Harari	Harper Perennial
7	978-0-7432-4722-4	The Da Vinci Code	Mystery	8.0	yes	Dan Brown	Doubleday
8	978-0-09-957807-9	A Game of Thrones	Fantasy	7.5	yes	George R.R. Martin	Bantam
33	978-0-7432-4722-5	Angels & Demons	Mystery	7.5	yes	Dan Brown	Doubleday

Task 12 — Books Not Yet Returned

Goal: Find every loan that hasn't been returned yet.

💡 **Concept — anti-join with `merge(how="left")`** : a *left merge* keeps every row from `issued_status` , filling in `NaN` where there's no matching row in `return_status` . Filtering for `NaN` in the return table's key column gives us exactly the loans with no return record — an "anti-join."

In [26]:

```
loans_with_returns = issued_status.merge(return_status, on="issued_id", how="left")

task12 = loans_with_returns[loans_with_returns["return_id"].isna()]
task12[["issued_id", "issued_member_id", "issued_book_name", "issued_date", "issued_emp_id"]]
```

Out[26]:

	issued_id	issued_member_id	issued_book_name	issued_date	issued_emp_id
15	IS121	C102	The Shining	2024-03-25	E109
16	IS122	C102	Fahrenheit 451	2024-03-26	E109
17	IS123	C103	Dune	2024-03-27	E109
18	IS124	C104	Where the Wild Things Are	2024-03-28	E110
19	IS125	C105	The Kite Runner	2024-03-29	E110
20	IS126	C105	Charlotte's Web	2024-03-30	E110
21	IS127	C105	Beloved	2024-03-31	E110
22	IS128	C105	A Tale of Two Cities	2024-04-01	E110
23	IS129	C105	The Stand	2024-04-02	E110
24	IS130	C106	Moby Dick	2024-04-03	E101
25	IS131	C106	To Kill a Mockingbird	2024-04-04	E101
26	IS132	C106	The Hobbit	2024-04-05	E106
27	IS133	C107	Angels & Demons	2024-04-06	E106
28	IS134	C107	The Diary of a Young Girl	2024-04-07	E106
29	IS135	C107	Sapiens: A Brief History of Humankind	2024-04-08	E108
30	IS136	C107	1491: New Revelations of the Americas Before C...	2024-04-09	E102
31	IS137	C107	The Catcher in the Rye	2024-04-10	E103
32	IS138	C108	The Great Gatsby	2024-04-11	E104
33	IS139	C109	Harry Potter and the Sorcerers Stone	2024-04-12	E105

Task 13 — Members with Overdue Books (More Than 30 Days)

Goal: Assuming a 30-day loan period, find every member currently holding an overdue, unreturned book, and how many days overdue it is.

💡 **Concept — date arithmetic:** subtracting two datetime columns gives a `Timedelta` ; `.dt.days` converts that into a plain integer number of days.

```
In [27]: overdue = (
    issued_status.merge(members, left_on="issued_member_id", right_on="member_id")
    .merge(books, left_on="issued_book_isbn", right_on="isbn")
    .merge(return_status, on="issued_id", how="left")
)
overdue = overdue[overdue["return_date"].isna()].copy()
overdue["days_since_issued"] = (today - overdue["issued_date"]).dt.days

task13 = (
    overdue[overdue["days_since_issued"] > 30]
    [["issued_member_id", "member_name", "book_title", "issued_date", "days_since_issued"]]
    .sort_values("days_since_issued", ascending=False)
)
task13
```

Out[27]:

	issued_member_id	member_name	book_title	issued_date	days_since_issued
15	C102	Bob Smith	The Shining	2024-03-25	904
16	C102	Bob Smith	Fahrenheit 451	2024-03-26	903
17	C103	Carol Davis	Dune	2024-03-27	902
18	C104	Dave Wilson	Where the Wild Things Are	2024-03-28	901
19	C105	Eve Brown	The Kite Runner	2024-03-29	900
20	C105	Eve Brown	Charlotte's Web	2024-03-30	899
21	C105	Eve Brown	Beloved	2024-03-31	898
22	C105	Eve Brown	A Tale of Two Cities	2024-04-01	897
23	C105	Eve Brown	The Stand	2024-04-02	896
24	C106	Frank Thomas	Moby Dick	2024-04-03	895
25	C106	Frank Thomas	To Kill a Mockingbird	2024-04-04	894
26	C106	Frank Thomas	The Hobbit	2024-04-05	893
27	C107	Grace Taylor	Angels & Demons	2024-04-06	892
28	C107	Grace Taylor	The Diary of a Young Girl	2024-04-07	891
29	C107	Grace Taylor	Sapiens: A Brief History of Humankind	2024-04-08	890
30	C107	Grace Taylor	1491: New Revelations of the Americas Before C...	2024-04-09	889
31	C107	Grace Taylor	The Catcher in the Rye	2024-04-10	888
32	C108	Henry Anderson	The Great Gatsby	2024-04-11	887
33	C109	Ivy Martinez	Harry Potter and the Sorcerers Stone	2024-04-12	886

Task 14 — Process a Book Return (as a reusable function)

Goal: Write a small piece of reusable logic that, given a `return_id`, `issued_id`, and the book's condition, records the return and marks the book as available again.

To record the book's *condition* on return, we need a place to store it. Let's check whether `return_status` already tracks that.

```
In [28]: print("book_quality" in return_status.columns)
return_status.head()
```

False

Out[28]:

	return_id	issued_id	return_book_name	return_date	return_book_isbn
0	RS101	IS101	NaN	2023-06-06	NaN
1	RS102	IS105	NaN	2023-06-07	NaN
2	RS103	IS103	NaN	2023-08-07	NaN
3	RS104	IS106	NaN	2024-05-01	NaN
4	RS105	IS107	NaN	2024-05-03	NaN

It doesn't exist yet. So before we can write the return-processing logic, we need to add that column to `return_status` — the pandas equivalent of a database's `ALTER TABLE ... ADD COLUMN`.

We'll default every existing return to `'Good'`, then go back and mark the handful we know were returned damaged

(based on the library's notes) using **boolean indexing** — selecting rows where a condition is `True` and assigning a value only to those rows.

```
In [29]: return_status["book_quality"] = "Good"

damaged_ids = ["IS109", "IS112", "IS115"] # returns the library's notes flagged as damaged
return_status.loc[return_status["issued_id"].isin(damaged_ids), "book_quality"] = "Damaged"

return_status[return_status["book_quality"] == "Damaged"]
```

```
Out[29]:
```

	return_id	issued_id	return_book_name	return_date	return_book_isbn	book_quality
6	RS107	IS109	NaN	2024-05-07	NaN	Damaged
9	RS110	IS112	NaN	2024-05-13	NaN	Damaged
12	RS113	IS115	NaN	2024-05-19	NaN	Damaged

Now that `return_status` can actually track book condition, we can write the reusable return-processing logic itself.

💡 **Concept — wrapping repeated logic in a function:** instead of writing this out by hand for every return, we define it once as a Python function. It takes the tables as input, updates them, and returns the updated versions — this is the same idea as a *stored procedure* in a database, just written in plain Python.

```
In [30]: def process_return(return_id, issued_id, book_quality, books_df, issued_df, returns_df):
    """Record a book return and mark the book as available again."""
    loan = issued_df[issued_df["issued_id"] == issued_id].iloc[0]
    isbn = loan["issued_book_isbn"]
    book_name = loan["issued_book_name"]

    new_return = pd.DataFrame([
        {
            "return_id": return_id,
            "issued_id": issued_id,
            "return_book_name": book_name,
            "return_date": pd.Timestamp.now().normalize(),
            "return_book_isbn": isbn,
            "book_quality": book_quality,
        }
    ])
    returns_df = pd.concat([returns_df, new_return], ignore_index=True)

    books_df.loc[books_df["isbn"] == isbn, "status"] = "yes"

    print(f"Recorded return for '{book_name}' – condition: {book_quality}")
    return books_df, returns_df

# Try it out on a currently-unreturned loan
books, return_status = process_return("RS150", "IS121", "Good", books, issued_status, return_status)
return_status.tail(3)
```

Recorded return for 'The Shining' – condition: Good

```
Out[30]:
```

	return_id	issued_id	return_book_name	return_date	return_book_isbn	book_quality
16	RS117	IS119	NaN	2024-05-27	NaN	Good
17	RS118	IS120	NaN	2024-05-29	NaN	Good
18	RS150	IS121	The Shining	2026-09-15	978-0-385-33312-0	Good

Step 6 — Advanced Analysis

Task 15 — Branch Performance Report

Goal: For each branch, show how many books were issued, how many were returned, and the total revenue generated — a one-stop report for management.

```
In [31]: branch_activity = (
    issued_status.merge(employees, left_on="issued_emp_id", right_on="emp_id")
    .merge(branch, on="branch_id")
    .merge(return_status, on="issued_id", how="left")
    .merge(books, left_on="issued_book_isbn", right_on="isbn")
)

branch_report = (
    branch_activity.groupby("branch_id")
    .agg(
        books_issued=("issued_id", "count"),
        books_returned=("return_id", "count"),
        total_revenue=("rental_price", "sum"),
    )
)
```

```
)
.reset_index()
.sort_values("total_revenue", ascending=False)
)
branch_report
```

```
Out[31]:
```

	branch_id	books_issued	books_returned	total_revenue
0	B001	17	9	111.5
4	B005	9	3	50.0
3	B004	4	3	26.5
2	B003	3	1	20.0
1	B002	1	0	6.5

Task 16 — Active Members (Issued a Book Recently)

Goal: Find members who have issued at least one book within the last 2 months.

Same idea as Task 9 — let's just write the filter first.

```
In [32]: cutoff = today - pd.DateOffset(months=2)

active_member_ids = issued_status.loc[issued_status["issued_date"] >= cutoff, "issued_member_id"].unique()
active_members = members[members["member_id"].isin(active_member_ids)]
active_members
```

```
Out[32]:
```

member_id	member_name	member_address	reg_date
-----------	-------------	----------------	----------

Empty again, for the same reason as Task 9: our `issued_status` records are all from March–April 2024, so nothing falls within 2 months of today's real date. This time, rather than inventing new loan records, it makes more sense to treat the **most recent activity actually in the dataset** as our reference point — "active as of the last time this library was used," which is a completely normal thing to do when analyzing a fixed historical extract. In a live system reading fresh data every day, you'd just use `today` directly, as we did above.

```
In [33]: reference_date = issued_status["issued_date"].max()
cutoff = reference_date - pd.DateOffset(months=2)

active_member_ids = issued_status.loc[issued_status["issued_date"] >= cutoff, "issued_member_id"].unique()
active_members = members[members["member_id"].isin(active_member_ids)]

print(f"Reference date used: {reference_date.date()} | cutoff: {cutoff.date()}")
active_members
```

Reference date used: 2024-04-12 | cutoff: 2024-02-12

```
Out[33]:
```

	member_id	member_name	member_address	reg_date
0	C101	Alice Johnson	123 Main St	2021-05-15
1	C102	Bob Smith	456 Elm St	2021-06-20
2	C103	Carol Davis	125 Oak St	2021-07-10
3	C104	Dave Wilson	567 Pine St	2021-08-05
4	C105	Eve Brown	890 Maple St	2021-09-25
5	C106	Frank Thomas	234 Cedar St	2021-10-15
6	C107	Grace Taylor	345 Walnut St	2021-11-20
7	C108	Henry Anderson	456 Birch St	2021-12-10
8	C109	Ivy Martinez	567 Oak St	2022-01-05
9	C110	Jack Wilson	678 Pine St	2022-02-25

Task 17 — Top 3 Employees by Books Issued

Goal: Find the three employees who have processed the most loans, along with their branch.

🔗 **Concept** — `sort_values()` + `.head()` : sort the aggregated counts from highest to lowest, then take the top rows — the pandas equivalent of `ORDER BY ... DESC LIMIT 3`.

```
In [34]: task17 = (
    issued_status.merge(employees, left_on="issued_emp_id", right_on="emp_id")
```

```

        .merge(branch, on="branch_id")
        .groupby(["emp_id", "emp_name", "branch_id"])
        .size()
        .reset_index(name="books_processed")
        .sort_values("books_processed", ascending=False)
        .head(3)
    )
task17

```

Out[34]:

	emp_id	emp_name	branch_id	books_processed
9	E110	Laura Martinez	B005	6
5	E106	Michelle Ramirez	B001	6
7	E108	Jessica Taylor	B004	4

Task 18 — Members Who Repeatedly Return Damaged Books

Goal: Flag members who have returned more than 2 books marked 'Damaged' — potential "high-risk" borrowers worth following up with.

```

In [35]: damaged_loans = (
    issued_status.merge(books, left_on="issued_book_isbn", right_on="isbn")
    .merge(members, left_on="issued_member_id", right_on="member_id")
    .merge(return_status, on="issued_id")
)
damaged_loans = damaged_loans[damaged_loans["book_quality"] == "Damaged"]

damage_counts = (
    damaged_loans.groupby(["member_id", "member_name"])
    .size()
    .reset_index(name="damaged_returns")
)

print("All members with at least one damaged return:")
display(damage_counts)

task18 = damage_counts[damage_counts["damaged_returns"] > 2]
print("\nMembers exceeding the 2-damaged-book threshold:")
task18

```

All members with at least one damaged return:

	member_id	member_name	damaged_returns
0	C109	Ivy Martinez	3

Members exceeding the 2-damaged-book threshold:

Out[35]:

	member_id	member_name	damaged_returns
0	C109	Ivy Martinez	3

Task 19 — Issue a Book (as a reusable function)

Goal: Write a function that issues a book to a member, but only if it's currently available (`status == 'yes'`). If it's unavailable, it should print a clear message instead.

This mirrors Task 14 — the same idea of wrapping a business rule ("check availability, then act") into a function you can call again and again.

```

In [36]: def issue_book(issued_id, member_id, isbn, emp_id, books_df, issued_df):
    """Issue a book to a member if it is currently available."""
    match = books_df.loc[books_df["isbn"] == isbn, "status"]
    if match.empty:
        print(f"No book found with ISBN {isbn}")
        return books_df, issued_df

    if match.iloc[0] == "yes":
        book_title = books_df.loc[books_df["isbn"] == isbn, "book_title"].iloc[0]
        new_loan = pd.DataFrame([
            "issued_id": issued_id,
            "issued_member_id": member_id,
            "issued_book_name": book_title,
            "issued_date": pd.Timestamp.now().normalize(),
            "issued_book_isbn": isbn,
            "issued_emp_id": emp_id,
        ])
        issued_df = pd.concat([issued_df, new_loan], ignore_index=True)

```

```

books_df.loc[books_df["isbn"] == isbn, "status"] = "no"
print(f'"{book_title}" issued successfully to member {member_id}')
else:
    print(f"Sorry – the book with ISBN {isbn} is currently unavailable")

return books_df, issued_df

# Try issuing a book that IS available
books, issued_status = issue_book("IS200", "C108", "978-1-60129-456-2", "E104", books, issued_status)

# Try issuing a book that is NOT available (status was flipped to 'no' by the call above)
books, issued_status = issue_book("IS201", "C108", "978-1-60129-456-2", "E104", books, issued_status)

```

'To Kill a Mockingbird' issued successfully to member C108
 Sorry – the book with ISBN 978-1-60129-456-2 is currently unavailable

Task 20 — Overdue Fines Report

Goal: Build a summary table of overdue fines: for every member with unreturned books overdue by more than 30 days, show how many overdue books they have and their total fine, charged at **\$0.50/day**.

This combines everything we've used so far: merging, filtering, date math, and aggregation.

```

In [37]: overdue_loans = issued_status.merge(return_status, on="issued_id", how="left")
overdue_loans = overdue_loans[overdue_loans["return_id"].isna()].copy()
overdue_loans["days_overdue"] = (today - overdue_loans["issued_date"]).dt.days
overdue_loans = overdue_loans[overdue_loans["days_overdue"] > 30]

fines_report = (
    overdue_loans.groupby("issued_member_id")
        .agg(
            overdue_books=("issued_id", "count"),
            total_fine=("days_overdue", lambda days: (days * 0.50).sum()),
        )
        .reset_index()
        .rename(columns={"issued_member_id": "member_id"})
        .sort_values("total_fine", ascending=False)
)
fines_report

```

```

Out[37]:
   member_id  overdue_books  total_fine
3         C105              5      2245.0
5         C107              5      2225.0
4         C106              3      1341.0
0         C102              1       451.5
1         C103              1       451.0
2         C104              1       450.5
6         C108              1       443.5
7         C109              1       443.0

```

Wrap-Up & Next Steps

You've now rebuilt the full Library Management System analysis in Python:

- **Loaded and cleaned** six related tables with pandas
- **CRUD operations:** added, updated, deleted, and retrieved records
- **Built derived tables** from raw data (book issue counts, expensive books, branch reports, fines)
- **Answered 20 real analytical questions** using filtering, merging, grouping, and date arithmetic
- **Wrapped repeatable business logic** (`process_return` , `issue_book`) into reusable functions

Ideas to extend this project further (good for a portfolio):

- Add charts with `matplotlib` or `seaborn` (e.g. revenue by category, top books by issue count)
- Export any of the summary tables with `.to_csv("report.csv", index=False)`
- Wrap the whole thing in a small script with a menu (issue a book, process a return, run a report)
- Connect this notebook directly to a live database using `sqlalchemy` + `pd.read_sql()` instead of CSVs

Key pandas skills demonstrated here (worth mentioning on a resume):

`pd.read_csv` , boolean indexing & `.loc` , `pd.concat` (insert), `merge` (join), `groupby` / `agg` (aggregate), `datetime`

handling (`pd.to_datetime`, date subtraction), and writing small reusable functions for business logic.